Global Convergence for Nash Equilibria: Implementing the Herings & Peeters Differentiable Homotopy in Gambit

Personal Details
Name: Andrés Fernández Cervell
University: Universidad de Granada, Spain.
Email: andresfernandezcervell@gmail.com
Linkedin: Andrés Fernández Cervell | LinkedIn

Abstract

The predictive power of game-theoretic analysis in n-person games is frequently undermined by the vast multiplicity of Nash equilibria. Selecting the most appropriate solution, according to a certain context, has been a fundamental challenge in Game Theory, theoretically addressed by Harsanyi and Selten's linear tracing procedure. However, computing this procedure remained difficult until P. Jean-Jacques Herings and Ronald J.A.P. Peeters introduced a globally convergent, everywhere differentiable homotopy algorithm.

This 175-hour project will implement the globally convergent algorithm proposed by Herings & Peeters (2001) into the Gambit C++ core and expose it via the PyGambit API. The C++ implementation will follow the algorithm explained in section 4 of Herings & Peeters paper. It basically consists of a transformation of variables that makes the path to follow differentiable. Therefore, it is possible to trace it using precise and efficient methods, taking into consideration a prior condition.

About me

I am a second year student currently pursuing a Double Degree in Mathematics and Computer Science. This background allows me to bridge the gap between abstract mathematical theory and high-performance software engineering. My experience includes:

- **Contribution to Gambit:** I am already familiar with the Gambit codebase and curves-following methods. I worked on resolving Issue #492, where I developed a mathematical report and C++ implementation using Sard's Theorem to resolve numerical stability issues in Nash equilibrium solvers using iterative methods.
  - Fix #492.
  - Full Technical Report.
- **Open Source & Python API Experience:** Beyond low-level C++, I have proven experience designing modern, interoperable Python architectures. I successfully developed a Proof of Concept for the scikit-bio library, refactoring their internal metrics to support the Python Array API standard (NEP 47). This experience ensures I can cleanly integrate the new C++ solver into the PyGambit ecosystem.
  - scikit-bio array API refactor (Proof of Concept)

- **Academic Background:** a current GPA of 8.0/10.0 provides me with a strong technical foundation in fields such as Linear Algebra, Topology, Numerical Methods, Algorithms and Data Structures. Experience with Python, C/C++, Java and Ruby. Award-winning communication skills (winner of Provincial Debate Tournament) and academic excellence, as well as fluency in Spanish (native), English (C1- Cambridge-Grade A) and French (B2-DELF).

Motivation & Impact

The Bottleneck: Equilibrium Multiplicity and Algorithmic Blindness

As models in economics, cybersecurity, and Multi-Agent AI scale, the number of potential Nash equilibria grows exponentially. The current Gambit catalog offers excellent tools for specific domains, but they face limitations in general n-person games:

- `lp`, `lcp` and `enummixed` only work for two-player game.
- `enumpure` will only find pure-strategy equilibria of a game.
- `enumpoly` computes all equilibria of a game, making it inefficient for large games. Besides, it does not give any information about which equilibrium might be the most realistic.
- `liap` is not globally convergent.
- `simpdiv` uses a simplicial subdivision approach. Quoting Herings & Peeters: " A problem of these algorithms is that they calculate only an approximation of a sample Nash equilibrium and do not bother about the game-theoretic underpinning of the calculated equilibrium."
- `ipa` and `gnm` use a global Newton method, but take as a parameter a limited prior (the most frequently played strategy must be unique).
- `logit` does not take subjective priors into consideration, critical in real world information models.

Impact and Community Benefits

This project will introduce a solver that computes the most appropriate Nash Equilibrium according to a prior condition. It will be done using an efficient differentiable tracking, that will reuse the robust path-tracking architecture from `logit_solve` (after solving Issue #492). Furthermore, I will update PyGambit so that this new method is available for data scientists and AI researchers.

Technical Methodology

The implementation will be highly modular, prioritizing code reuse from Gambit's existing infrastructure. The development is divided into five main blocks:

- **Securing the Path-Tracking Foundation (Issue #492):** Before building the Herings & Peeters homotopy, it is imperative to have a completely robust path-tracking foundation. Based on mentor feedback regarding my previous work on the `logit_solve` path-following methods, I will first finalize Issue #492. This includes implementing a configurable perturbation system and adding a bifurcation logging mechanism so users can explore unstable points in detail.

- **Variable Transformation & System Definition:** I will implement the variable transformation proposed by Herings & Peeters: $\sigma_j^i(\alpha) = [\max\{0, \alpha_j^i\}]^2$ and $\lambda_j^i(\alpha) = [\max\{0, -\alpha_j^i\}]^2$. This converts the problem into an everywhere differentiable C^1 manifold. I will program the new algebraic homotopy function $\mathcal{H}(t, \alpha, \mu) = 0$. Furthermore, the initialization logic will be built to compute the exact starting point at $t = 0$ by mapping the user-provided Prior into the transformed state variables.

- **Analytical Jacobian & Path Tracking Engine:** Having deeply analyzed Gambit's path-tracking mechanics while resolving Issue #492 for `logit_solve`, I will adapt its existing predictor-corrector architecture. My work will focus on coding the analytical Jacobian matrix of $\mathcal{H}$ required for the Euler predictor steps and the Newton-Raphson corrector steps. I will adapt the dynamic step-size logic to trace the parameter t strictly within the $[0, 1]$ interval, extracting the Nash Equilibrium when $t = 1$ is reached. Furthermore, as suggested by my mentors, I will refactor the existing Allgower-Georg path-tracking code into an abstract interface. This decoupling ensures that both the Logit solver and the new H&P homotopy can share the same tracking engine, while making it easier for future contributors to drop in alternative solvers (such as HOMPACK) without breaking existing implementations.

- **PyGambit API Integration:** To make the algorithm accessible to the data science and AI communities, I will write the necessary bindings to expose the C++ solver to Python.

- **Testing & Performance Benchmarking:** I will implement a comprehensive test suite using `pytest`. The validation will check that the new method successfully converges to the theoretically correct equilibrium on standard catalog games. I will benchmark its execution time and memory footprint against other Gambit Nash Equilibrium algorithms to empirically demonstrate the performance gains on larger n-person games.

Potential Challenges & Mitigations

While the mathematical theory guarantees global convergence, implementing smooth path-following methods in floating-point arithmetic presents inherent engineering challenges:

- **Challenge: Numerical Instability.** Floating-point inaccuracies can cause the Newton-Raphson corrector to diverge or leave to unexpected behaviors.

- *Mitigation:* I have already successfully tackled numerical stability and multidimensional symmetry breaking issues using Sard's Theorem in my previous work for Gambit (Issue #492). Securing this foundation guarantees a battle-tested predictor-corrector engine before introducing the new homotopy.

- **Challenge: C++ / Python Interoperability.** Safely passing Priors from Python's high-level environment to low-level C++ arrays can lead to memory leaks or type mismatches.
- *Mitigation:* Rather than building the bridge from scratch, I will strictly mirror the proven wrapper architectures already used by Gambit's other solvers, isolating the conversion logic and testing it heavily with `pytest` before full integration.

Timeline

I am able to work every day of the week, with a flexible schedule to program chats and discussions with my mentors to ensure a high quality project expected to take around 175 hours.

Timeframe	Tasks & Deliverables
Until May 24	Communication with mentors, familiarization with the code and the community, the documentation and test system used. Studying the mathematics of the algorithm.
May 24 - June 25 Part- time dedication, since I will be in class and doing my final exams. I will focus on solving the Issue #492 and establishing the new mathematical core in C++.	
Weeks 1-2 (May 25 - June 7)	Finalize Issue #492. Implement the configurable perturbation system and the bifurcation logging mechanism based on recent mentor feedback. Ensure the predictor-corrector engine is robust and merged before building upon it.
Weeks 3-4 (June 8 - June 25)	Implement the H&P variable transformation logic (σ and λ) and the algebraic $\mathcal{H}(t, \alpha, \mu)$ evaluation functions. Code the analytical Jacobian matrix in C++ and write isolated unit tests.
Week 5 (June 26 - July 2)	Vacations (all PRs from previous works will be submitted for review beforehand so mentors have material to evaluate during my absence.)
July 3 - August 24: Full time dedication. I will have no more responsibilities, and will only be off for some days of vacations (not confirmed yet)	
Week 6 (July 3 - July 10)	Code, documentation and revisions. Preparing Midterm Evaluation.
Weeks 7-8-9 (July 11 - August 3)	Abstract the Allgower and Georg´s path-following method. Hook the H&P system into the newly robust predictor-corrector engine. Extensive debugging of Newton convergence.
Weeks 10-11-12 (August 3 - August 24)	Develop the Python bindings, write the `pytest` suite, run comparative benchmarks against other Gambit NE algorithms. Final documentation and preparing Final Evaluation.